RL for Autonomous Flight Control

Dosquet Pierre, Van de Vyver Eri

May 6, 2026

0 Introduction

This project uses deep reinforcement learning algorithms for quadrotor and fixed-wing drone control across three tasks of increasing difficulty. **QuadX-Hover-v4** that requires the agent to maintain a stable hover at a fixed point under a dense reward signal. **QuadX-Waypoints-v4** that introduces a sparse reward environment where the agent must reach up to four randomly placed targets in a 150 m flight dome. Finally, **MAFixedwingDogfightEnvV2** that is a 1vs1 adversarial combat task.

Two algorithms have been trained and compared: Proximal Policy Optimization (PPO, on-policy) and Soft Actor-Critic (SAC, off-policy). Their performance have been studied for to verify if one approach is better than the other across tasks or whether the right choice depends on reward density and task structure. All experiments are tracked on Weights & Biases. The public project is available at `https://wandb.ai/pdosquet-ulg/pyflyt-rl`.

1 MDP Formalisation

1.1 QuadX-Hover-v4 - Stabilisation Task

- **State space** $\mathcal{S}$: A flat vector of size 21 concatenated in the following order:
  - **Angular velocities** (p, q, r): body-frame roll, pitch, and yaw rates (3 values).
  - **Attitude**: Quaternion orientation (q_w, q_x, q_y, q_z) (4 values).
  - **Linear velocities** (v_x, v_y, v_z): body-frame translational velocities (3 values).
  - **Absolute position** (x, y, z): world-frame position of the drone (3 values).
  - **Previous action**: the last action sent to the motors (4 values).
  - **Auxiliary state**: motor RPM and other low-level sensor readings from the simulator (4 values).
- **Action space** $\mathcal{A}$: A continuous vector $a \in [-1, 1]$ of size 4. The physical semantics of the action vector depend on the flight_mode:
  - **Mode 0 (Hard)**: $a = (p, q, r, T)$, representing target angular velocities (roll, pitch, and yaw rates) and the collective thrust.
  - **Mode 6 (Easy)**: $a = (\dot{x}, \dot{y}, r, \dot{z})$, representing ground velocities, yaw rate, and vertical velocity.
- **Transition model** $\mathcal{P}$: Transition are handled by the simulator that maintains 120 Hz, the policy operates at a frequency of 30 Hz.

- **Reward function** The reward r_t received at each time step t is defined by the following function:

$$r_t = \begin{cases} -100.0 & \text{if crash or out-of-bounds} \\ 0.9 - \|pos\_error\| - \|tilt\_error\| - 0.01 \cdot \dot{\psi}^2 & \text{otherwise} \end{cases} \tag{1}$$

  - **Linear Distance Penalty** $\|pos\_error\|$: Euclidean distance between the drone and the fixed hover target at $(0, 0, 1)$ m thus at 1m from the ground.
  - **Angular Tilt Penalty** $\|tilt\_error\|$: Euclidean norm of the roll and pitch angles, encouraging the drone to stay level.
  - **Yaw Rate Penalty** $0.01 \cdot \dot{\psi}^2$: Squared yaw rate discourages rapid spinning. Note the quadratic penalty.
  - **Base step cost**: Each step starts at -0.1 before the $+1.0$ bonus is added, giving a net constant of 0.9 in the non-terminal case.
  - **Terminal Penalty**: Immediately overrides the reward with -100 upon crash or leaving the flight dome.

- **Episode termination**: The episode ends if the drone crashes (due to excessive tilt or going out of bounds) or reaches a maximum duration of a fixed amount of steps.

- **Discount factor** γ: A value of $\gamma \in \{0.98, 0.99\}$ is used to balance immediate stabilisation with long hovering objective.

1.2 QuadX-Waypoints-v4 - Navigation Task

- **State space**: Same 21-dimensional attitude vector as hover, concatenated with body-frame relative displacement vectors $(\Delta x_i, \Delta y_i, \Delta z_i)$ for each remaining waypoint, padded to a fixed number of 4 slots. With `num_targets=4` this gives a flat vector of size 33 ($21 + 4 \times 3$).

- **Action space**: The project evaluation environment uses mode 6 ($a = (\dot{x}, \dot{y}, r, \dot{z})$): ground-frame velocity commands and yaw rate, handled by the onboard PID cascade.

- **Reward function**: The reward r_t received at each time step t is defined by the following piecewise function:

$$r_t = \begin{cases} +100.0 & \text{if waypoint reached} \\ -100.0 & \text{if crash or out-of-bounds} \\ -0.1 + \max(3.0 \cdot \text{progress},\, 0) + \dfrac{0.1}{\text{dist}} - 0.01 \cdot \dot{\psi}^2 & \text{otherwise} \end{cases} \tag{2}$$

  - **progress**: Signed advance toward the *current* target waypoint since the last step; clamped to 0 when the drone moves away.
  - **dist**: Euclidean distance to the current target waypoint.
  - $\dot{\psi}^2$: Squared yaw rate, penalising rapid spinning.
  - **Potential-based shaping**: In all our experiments, a `WaypointDistanceShaping` wrapper adds $\varphi(s') - \varphi(s)$ at each step, where $\varphi(s) = -0.2 \cdot \|\Delta_{\text{next}}\|$. This creates a continuous gradient toward the next waypoint without altering the optimal policy.

- **Episode termination**: Crash or timeout after $30 \times 120 = 3600$ steps (120 s at 30 Hz).

1.3 Dogfight - MAFixedwingDogfightEnvV2

- **State space**:
  - **Self State (21 values)**: Includes attitude (angular velocity, angular position, linear velocity, linear position), auxiliary flight data as before but also current health and the previous action taken.
  - **Opponent State (14 values)**: Relative attitude transformed to the agent's body frame, opponent health and a team flag indicating friend or foe.
- **Action space**: A continuous vector $a \in [-1, 1]$ of size 4 representing fixed-wing control surfaces: throttle, elevator (pitch), rudder (yaw), and aileron (roll). This simplifies the policy's job, it only decides where to point the aircraft (attitude targets), while PyFlyt's internal controllers handle how to move the control surfaces to achieve that.
- **Reward function**: The reward r_t is a sum of engagement and boundary components, subject to terminal overrides:

$$r_t = \begin{cases} -1000.0 & \text{if collision or out-of-bounds} \\ +300.0 & \text{if team win} \\ r_{engagement} + r_{boundary} & \text{otherwise} \end{cases} \tag{3}$$

  - **Engagement Rewards** ($r_{\text{engagement}}$): Comprises rewards for closing distance to the enemy, improving engagement angles, achieving a firing position, scoring hits and team cooperation (not applicable in our situation since this is a 1vs1).
  - **Boundary Rewards** (r_{boundary}): Includes a hyperbolic tangent function (tanh) based on altitude to maintain flight, a penalty for approaching flight dome edges and a collision avoidance penalty for proximity to other aircraft.
- **Episode termination**: An agent is destroyed, or 60 s timeout at 30 Hz.

2 Algorithms

We train and compare two families of algorithms: Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC). SAC is used exclusively through Stable-Baselines3 (SB3) via RL Zoo 3. PPO is used in two forms: the SB3 implementation via RL Zoo 3, and an implementation attempt for the hover task to verify that we can reach similar behaviour with a minimal codebase.

2.1 Proximal Policy Optimization (PPO)

PPO is an on-policy actor-critic algorithm. At each iteration, a batch of environment interactions (a rollout) is collected under the current policy π_θ. The policy is then updated by maximising the clipped surrogate objective, which uses the probability ratio between the new and old policies:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

The advantage $\hat{A}_t$ is estimated via the Generalised Advantage Estimate (GAE), which trades off bias and variance in the credit-assignment signal.

We also implemented a PPO algorithm version, which differs from the SB3 version in following ways:

- **ActorNetwork**: 2-layer MLP (tanh activations, 64 units), Gaussian head with learned log-std parameter.
- **CriticNetwork**: 2-layer MLP value function (64 units). (The SB3 PPO instances use [256, 256] networks for both actors and critics.)
- **Full-batch updates**: Each epoch runs one gradient step over the entire rollout, whereas SB3 splits it into mini-batches of 64 transitions. This makes the custom implementation noisier.
- **Separate optimisers**: Actor and critic each have their own Adam instance (critic learning rate halved), whereas SB3 uses a single shared optimiser.
- **Fixed learning rate**: The custom implementation uses a constant 3×10^{-4}, the SB3 waypoints configs use a linear decay schedule.

Properties

- **On-policy**: Each batch is removed after the update, preventing the model to overfit a dataset but requiring many environment interactions.
- **Sample efficiency**: Limited. Transitions are discarded after each update, requiring many environment interactions. Multiple epochs over the same rollout partially mitigate this.
- **Exploration**: The stochastic Gaussian policy provides inherent exploration.
- **Stability**: Clipping prevents destructively large updates.

Why PPO: PPO is used as an on-policy baseline: its clipping mechanism provides stability on continuous action tasks without requiring a replay buffer and its on-policy updates prevent the Q-value overestimation that is detriment for off-policy methods on sparse rewards. This property is particularly helpful on the waypoints task. We also use PPO for the dogfight tournament: each rollout compare the current policy's behaviour to the current opponent and there is no buffer that could be negatively influenced by transitions collected against obsolete opponents methods.

2.2 Soft Actor-Critic (SAC)

SAC is an off-policy maximum-entropy actor-critic algorithm. It adds to the standard RL objective an entropy term:

$$J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}[r(s_t, a_t) + \alpha H(\pi(\cdot|s_t))]$$

where α is automatically tuned to maintain a target entropy level, preventing to converge to a suboptimal deterministic policy. SAC maintains a replay buffer and trains on randomly sampled past transitions, separating data collection from gradient updates and improving sample efficiency relative to on-policy methods.

SAC uses two Q-networks (clipped double-Q): independent critic networks that are trained simultaneously and the minimum of their predictions is used as the target. This avoidsto overestimate the Q-value that would otherwise make the policy greedy too early. Our SB3 SAC instances use a [256, 256] network architecture for both actor and critics.

Properties

- **Off-policy**: It uses a replay buffer, making it highly sample-efficient.
- **Update-to-Data (UTD) ratio**: controls how many gradient steps are taken per environment step. Higher UTD improves sample efficiency at the cost of compute.
- **Exploration**: automatic entropy tuning throughout training.

Why SAC: On the contrary of PPO, SAC is the off-policy baseline and is perfect for making a comparison between both algorithms. Its replay buffer makes it much more sample efficient than PPO on dense reward continuous action tasks, which is perfect for the hover task. The automatic entropy tuning adapts exploration to task difficulty, this could be an advantage when the required exploration level differs between hover and waypoints.

3 Experiments

3.1 Custom PPO implementation attempt

A custom PPO has also been implemented for the hover task. However, training the model did not result to any usable policy. The next section about results do thus not contain any result discussion about this custom implementation. The most interesting tries are yet still to be seen on the Weights & Biases project under the names ppo_hover_mode_*.

3.2 Experimental Setup

Libraries and frameworks: All experiments use Stable-Baselines3 (SB3) v2 with RL Zoo 3 for SAC and the SB3 PPO implementation. A custom PPO is also implemented in PyTorch for the hover task. Observation normalisation (for Waypoints) uses SB3's `VecNormalize` wrapper.

Experiment tracking: All runs are logged to Weights & Biases (WandB). Each run records hyperparameters, rollout metrics (`ep_rew_mean`, `ep_len_mean`), and training diagnostics (`critic_loss`, `ent_coef`, `explained_variance`, `entropy_loss`). Runs are grouped by task and algorithm family for comparison. Evaluation metrics (deterministic policy on a separate eval env) are excluded from reported results: when `VecNormalize` is used, the evaluation uses different normalization settings than the training environment. This causes evaluation scores to make the agent look worse than it actually is. The WandB project is publicly available at `https://wandb.ai/pdosquet-ulg/pyflyt-rl`.

Reproducibility: All runs are seeded via the seed argument passed to both env.reset() and the algorithm constructor. Hyperparameters are stored in YAML config files committed to the repository and automatically synced to WandB on run start. Hover sweeps use 2 seeds per configuration. Final waypoints models (PPO and SAC mode 0) use 3 seeds each. Curriculum intermediate stages are done using single seed exploratory runs.

3.3 Considerations for the Waypoints Task

Curriculum usage: Direct training on the full task (4 targets, 150 m dome) fails for both algorithms because the waypoint signal is never observed. Curriculum learning breaks the problem into stages of increasing difficulty, starting with 1 target in a small dome (30 m), where a waypoint occupies a much larger fraction of the reachable space and is hit early in training. Once the agent can reliably reach one target, the training is transferred to 2 targets and a larger dome, carrying over the learned policy as initialisation. This staged transfer ensures the agent to learn from a sparse reward signal as in Waypoints.

Mode 6 before mode 0: Mode 6 exposes simpler velocity commands. The onboard PID cascade handles attitude internally. The agent only needs to learn where to go. Mode 0 exposes angular velocity and thrust directly, forcing the agent to learn attitude stabilisation and navigation simultaneously. The curriculum is first developed and validated on mode 6 to isolate the navigation problem and the same strategy is then applied to mode 0 (used by the evaluation environment) once the curriculum design is established.

Reward shaping: Even with a small dome, the sparse waypoint reward alone provides no gradient between episodes: the agent either hits the target (rare) or does not. We therefore add potential-based shaping $\varphi(s') - \varphi(s)$, where $\varphi(s) = -\alpha\|\Delta_{\text{next}}\|$, creating a dense gradient at every step. Two shaping variants were compared. Sum shaping points toward all remaining targets simultaneously, attenuating the gradient into a weak signal rather than the immediate target. This method has not provided better results. Next-waypoint shaping (NWS, `WaypointDistanceShaping`) points exclusively toward the next target, giving a clearer navigation signal without changing the optimal policy. Switching from sum shaping to NWS was the largest improvement across all waypoints experiments.

4 Results

4.1 Hover Task

4.1.1 Experimental Sweep

We tested every combination of two discount factors γ (0.98 and 0.99) and two data collection settings (PPO: 2048 or 4096 steps per rollout; SAC: 3 or 4 gradient updates per step). Each configuration was run twice with different random seeds on both flight modes 0 and 6.

Mode	γ	Reward mean	Comments (both UTD $\in \{3, 4\}$)
0	0.98	≈1000	Stable. critic_loss $\to 0$ by 20k and ent_coef < 0.1 early.
0	0.99	*Fails*	Unstable ent_coef tuning under high observation variance. Fails for both UTD values. (*Fails* means that it did not converge within 100k steps)
6	0.98	≈1000	Converges reliably. PID cascade smooths observations, removing the γ instability seen on mode 0.
6	0.99	≈1000	Same as for $\gamma = 0.98$

Table 1: SAC mean rewards for the Hover task for 50k steps

Mode	γ	Reward mean	Comments (both $n_s \in \{2048, 4096\}$)
0	0.98	80-200 $\to$ 1100	Only 80-200 at 100k; full convergence at 500k. entropy_loss keeps climbing without plateau.
0	0.99	≈1100	Same as for $\gamma = 0.98$
6	0.98	≈1100	n_s=2048 collapses at 100k (effective horizon $1/(1-\gamma) = 50 \ll$ episode). n_s=4096 avoids collapse and converges faster.
6	0.99	≈1100	Nearly solved at 100k (≈800-900). entropy_loss stabilises at ≈ -3: policy commits to hover.

Table 2: PPO mean rewards for the Hover task for 500k steps. Full convergence to ≈1100 reward is reached at ≈200k steps on both modes.

Both algorithms converge to the same final reward (≈1100) on hover, but SAC is 5-10× more sample-efficient, reaching the optimal reward in 50k steps versus 500k for PPO. The flight mode does not affect final performance for either algorithm, but it affect well training dynamics: on mode 0, SAC's entropy tuning destabilises at $\gamma = 0.99$ and PPO's entropy loss never plateaus, while on mode 6 both are stable and converge cleanly. Neither UTD=4 (SAC) nor $n_s = 4096$ (PPO) are significantly better on hover. The task is simple enough for the baseline settings to be sufficient.

4.2 Waypoints Task

The waypoints task is harder than hover for two main reasons. First, the reward is sparse: a waypoint is only collected when the drone enters a 4 m radius sphere around a randomly placed target inside a 150 m dome. The probability that a random policy gets onto a waypoint is clearly small. An agent that has not yet learned to navigate will never receive the positive reward signal it needs to learn navigation. Second, the task is sequential: the agent must reach up to 4 targets in order, so failure to reach the first one prevents any learning about the rest.

The task has been approached in three phases: direct training to establish a baseline (§4.2.1), curriculum on mode 6 to explore curriculum design and reward shaping on the easier flight mode (§4.2.2), and curriculum on mode 0 for the evaluation environment (§4.2.3). Mode 6 ultimately did not scale to the full task (the best result stayed at 2 targets in a 30 m dome), but the key findings (NWS shaping, staged transfer) directly informed the mode 0 curriculum, which did succeed.

The curriculum consists in four stages of increasing difficulty. S1 uses a single target in a 30m dome with simple shaping. S2 adds a second target and introduces next waypoint shaping. S3 expands to 4 targets and a 60–100m dome. S4 reaches full difficulty with 4 targets in a 150m dome. Each stage loads the previous checkpoint, building from trivial navigation to the full waypoint task.

4.2.1 Baseline: Direct Training Without Curriculum

Both algorithms were trained directly on the full task (4 targets, 150 m dome, 4 m goal radius) without any curriculum.

Algo	Reward mean	Episode length mean	Steps	Comments
SAC	-300 to -400	100-3600	100k-500k	critic_loss ≈ 0, ent_coef_loss $= 0$: Q-network converged on survival. No waypoint signal reached.
PPO	≈ -300	≈ 3600 (ceiling)	500k	entropy_loss flat: policy frozen at survival plateau. Shaping coef=0.05 commits policy earlier but to wrong behaviour.

Table 3: Direct training on full waypoints task (mode 6).

Neither algorithm makes navigation progress. reward sparsity prevents any waypoint signal in a 150 m dome with 4 m goal radius.

4.2.2 Curriculum Learning on Mode 6

Two PPO training chains were developed, plus one SAC attempt. All start from a simplified task (1 target, small dome) and progressively increase difficulty.

Algo	Chain	Stage	Config	Reward mean	Episode length mean	Comments
PPO	1	S1	[64,64], γ=0.99, 1t, 30m, SumShaping	-260 to -300	>3000	No improvement over baseline. Sum shaping dilutes gradient toward centroid of both targets.
PPO	1	S2	same + NWS shaping	-100 to -162	1000-1250	Shaping change only: largest single improvement.
PPO	2	S1	[256,256], γ=0.995, 1M steps, 1t, 30m	+100	≈115	Strong anchor for stage 2.
PPO	2	S2	+SDE, obs-norm, NWS, 1.5M steps, 2t, 30m	**190-200**	≈40	Best mode 6 result obtained.
SAC	-	S1	γ=0.995, UTD=2, 500k, 1t, 30m	0 to -50	-	High variance; replay buffer mixes crashes with successes. No S2 run.

Table 4: Curriculum learning on mode 6 (PPO chains and SAC). NWS = next-waypoint-only shaping (`WaypointDistanceShaping`); SumShaping = `WaypointSumDistanceShaping`.

The progression $-300 \rightarrow -100 \rightarrow +190$ comes from NWS shaping and a stronger stage 1 anchor.

4.2.3 Mode 0 Navigation

All PPO stages share: n_s=4096, γ=0.995, linear lr decay, [256,256] network, `use_sde=True`, obs-norm, NWS shaping.

Algo	Stage	Config	Reward mean	Episode length mean	Comments
PPO	S3	3t, 30m dome, goal_r=10m, 1M steps	272	70	Reliable 3-target navigation in small dome.
PPO	Transfer	3t, 150m dome, goal_r=4m, 1M steps	800	315	First sustained navigation in the evaluation environment.
PPO	S4	4t, 150m dome, goal_r=4m, 1M steps, 3 seeds	800-900	650-812	Best seed ≈1000 (hover ceiling). 3 seeds confirm reproducibility.
SAC	-	Curriculum & direct (all configs)	≈ -100	<100	Transient spike to +100 then full collapse. Q-overestimation reproducible across all seeds.

Table 5: Mode 0 navigation results. PPO uses a staged curriculum; SAC fails on all configurations due to Q-value overestimation collapse.

4.3 Final Model Selection

Task	Algo	Configuration	Rew mean	Ep len mean	Status
Hover	SAC	g099_utd3_mode6	≈1100	400	Converged
Hover	PPO	g099_ns4096_500k_mode6	≈1100	400	Converged
Waypoints	PPO	s4_nws_sde_mode0 (3 seeds)	800-900	650-812	Best stable
Waypoints	SAC	g0995_utd3_500k_mode0	−100	50	Failed

Table 6: Final selected models per task and algorithm. Their respective performance can be found in Weights & Biases project.

The PPO waypoints model (`s4_nws_sde_mode0`) shows `explained_variance` of 0.6-0.8 and `entropy_loss` plateauing at ≈ -11, confirming genuine convergence: the value network fits the return distribution well and the policy maintains structured exploration across the 150m dome. The SAC hover model shows `critic_loss` ≈ 0 and `ent_coef_loss` ≈ 0, the cleanest convergence signature in the entire experiment set.

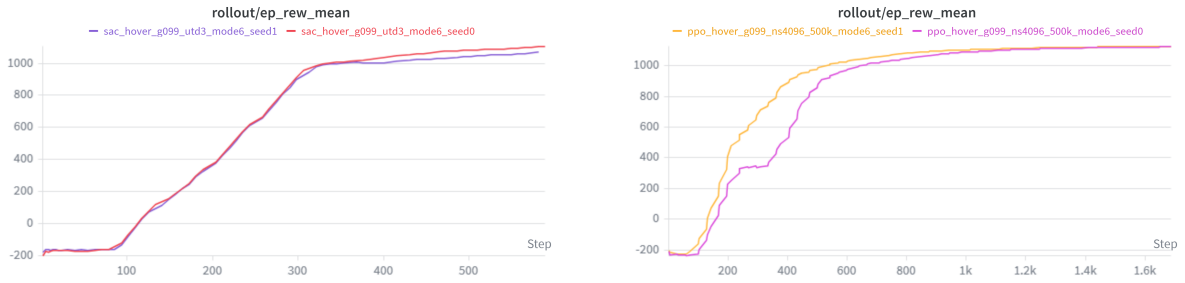

Figure 1: Final hover models ($\gamma = 0.99$, mode 6, 2 seeds each). **Left**: SAC (UTD=3) reaches ≈1000 reward by step 300 with near-identical seed behaviour. **Right**: PPO (n_s=4096) converges to the same ceiling but requires ≈1200 steps. Both plateau at ≈1100.

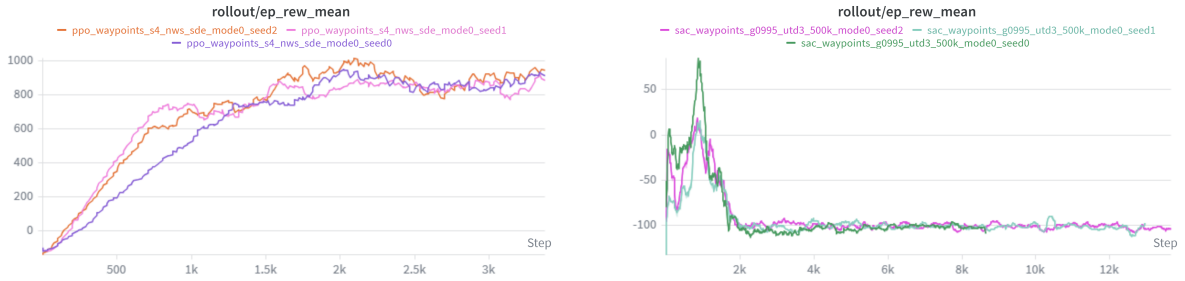

Figure 2: Final waypoints models (mode 0, 3 seeds each). **Left**: PPO stage 4 (n_s=4096, γ=0.995, SDE, NWS shaping) converges to 800-900 reward across all three seeds. **Right**: SAC (UTD=3, γ=0.995) shows the characteristic Q-overestimation failure: a transient spike to ≈+50-75 reward followed by complete collapse to ≈ -100 for all three seeds.

5 Analysis and Discussion

5.1 Algorithm Comparison

5.1.1 Hover task

Both PPO and SAC converge to the same final reward (≈1100) on hover, but differ in how they get there.

- **Sample efficiency:** SAC reaches the plateau within 50k steps and PPO first converges at ≈200k steps (within a 500k run). The replay buffer allows SAC to learn from past transitions without collecting new data, making it 5-10× more sample-efficient on this task.
- **Stability:** PPO with $n_s = 2048$ and $\gamma = 0.98$ collapses at 100k on mode 6 due to a too short effective horizon. SAC with $\gamma = 0.99$ fails on mode 0 due to unstable entropy coefficient tuning. Both issues disappear with the right hyperparameter choice.
- **Hyperparameter sensitivity:** SAC is sensitive to γ on mode 0 while PPO is sensitive to n_s. Neither UTD=4 nor $n_s = 4096$ offer a decisive advantage. The task is simple enough so that baseline settings are enough.

5.1.2 Waypoints task

The two algorithms act very differently on waypoints, since PPO succeeds where SAC fails. PPO with a staged curriculum reaches mean reward of 800-900 per episode on the full evaluation environment (4 targets, 150 m dome, mode 0) and SAC fails on every configuration tried (both direct training and curriculum) due to Q-value overestimation collapse.

SAC's off-policy replay buffer, which makes it efficient on hover, becomes a disadvantage on waypoints. With UTD=3, the critic receives three gradient updates per environment step. On a sparse task, the buffer is dominated by low-reward survival transitions with only a a few of rare waypoint episodes. The Q-network overestimates the value of those rare states, the policy becomes greedy toward them and the critic eventually diverges. The clipped double-Q mechanism is insufficient to prevent this at UTD=3 when the buffer distribution and the policy distribution diverge significantly.

PPO's on-policy means it only trains on transitions it just collected. There is no stale buffer to corrupt the value estimates. Each rollout reflects the current policy's actual behaviour, so the critic cannot overestimate unseen states. The cost is sample efficiency (PPO needs millions of steps), but given enough compute and a well-designed curriculum, it converges reliably.

5.2 Effect of Flight Mode on Waypoint Performance

Mode 0 is harder than mode 6 because it exposes angular velocity + thrust commands. To move, the agent must first tilt the drone (pitch/roll), stabilise and then be horizontal again. This require many movements for the drone to learn. Any policy that has not learned precise attitude control will crash before accumulating any navigation reward.

Mode 6 exposes ground-frame velocity commands directly. Moving from a point to another is a one-step command and the PID cascade handles the underlying attitude. This enables to learn faster how to get a higher reward.

Strangely, our best waypoints result was achieved on **mode 0** (PPO curriculum, 800-900 reward across 3 seeds) while mode 6 peaked at +190-200 on a 2-target, 30 m dome, a harder environment than the evaluation one. Mode 0 ultimately required, and benefited most from, curriculum learning: the staged transfer (small dome → full dome, 1 target → 4 targets) gave the agent time to develop attitude control before having to navigate. Mode 6 curriculum stalled because the PID abstraction, while easier to explore, also removes the low-level feedback that guides the agent toward stable flight in the full 150 m environment.

5.3 Failure Analysis

5.3.1 Crash Causes

Crashes occur for two distinct reasons depending on the flight mode and training phase.

On **mode 0**, the agent controls angular velocity and thrust directly. A policy that has not yet learned attitude stabilisation will issue unbalanced thrust commands, causing the drone to tip and accelerate into the ground. This is the main crash cause early in training, before the value network has seen enough trajectories to penalise tilt. The -100 terminal penalty discourages crashes, but in the first few thousand steps the policy is too random to react to the tilt signal before it is too late.

On **mode 6**, the PID cascade handles attitude, so crashes are rarer and occur later. The main cause is leaving the flight dome: the agent issues a velocity command that sends the drone toward a waypoint at the edge of the dome, overshoots and exits the boundary. This is more frequent on mode 0 (where the dome is 30m in curriculum stages) and on full-task mode 6 runs where the agent is moving aggressively without precise control of stopping distance.

In both modes, crashes also occur during policy collapse (see SAC Q-overestimation below): once the critic diverges, the policy issues arbitrary actions that rapidly destabilise the drone.

5.3.2 Waypoints Reward Plateau

The waypoints task shows a two phase failure:

- **Phase 1 (0-20k steps):** Random / early policy crashes frequently ($r \approx -600$ per episode).
- **Phase 2 (20k-500k steps):** Agent learns to avoid crashes. It mostly stays still, accumulating a survival reward ($r \approx -90$). The training curve rises from -600 to -300, then remains steady.

The main cause could be that the reward is too sparse. In a 150 m dome with a 4 m goal radius, the probability of a random trajectory hitting a waypoint is too small. The agent maximises its current reward signal but never receives the waypoint signal.

This problem can be tackled using reward shaping. Adding `WaypointDistanceShaping` with $\alpha_{\text{shape}} = 0.20$ creates a positive gradient pointing toward the next waypoint. With more compute (for example by running SAC at 1M steps with shaping ($\alpha = 0.20$) The training curve plateau could be overcome.

5.3.3 SAC Q-Value Overestimation Collapse

On the waypoints task, SAC with UTD=3 produced a characteristic failure across all seeds and configurations: a clear peak (up to $\approx +100$ reward on both mode 0 and mode 6 curriculum) followed by complete collapse to ≈ -100 survival reward. The mechanism is Q-value overestimation driven by the aggressive off-policy update ratio.

With UTD=3, the critic receives three gradient updates per environment step. On a sparse task, the replay buffer is dominated by survival transitions and contains rare high-reward waypoint episodes. The Q-network overestimates the value of those rare states: it assigns big values because they appear repeatedly in the buffer while being under-represented in the actual state distribution. The policy then becomes greedy toward those overestimated states, scores a high-reward episode, but the overestimation accumulates: each gradient step amplifies the error until the critic diverges and the policy resets to the safest known behaviour (hovering in place, `ep_rew_mean` ≈ -100).

SAC uses two Q-networks (clipped double-Q) precisely to counter overestimation, but this mechanism is insufficient at UTD=3 on sparse tasks where the buffer distribution and the policy distribution diverge significantly.

5.3.4 PPO Policy Collapse (Mode 6, $\gamma = 0.98$)

At 100k steps on mode 6, PPO with $\gamma = 0.98$ showed reward collapse near the end of training. The expected return horizon $1/(1-\gamma) = 50$ steps is substantially shorter than the episode length required for stable hover (hundreds of steps), so the value function cannot propagate to long-range stabilisation actions. With $n_s = 2048$, each rollout spans roughly 40 such horizons but the value bootstrap at the end of each segment is based on a severely discounted future, making hover appear less valuable than it is. At 500k steps, this issue disappears: the value function accumulates enough trajectory diversity to generalise across the episode despite the short discount horizon.

5.4 Compute and Time Limitations

5.4.1 Reward shaping ablation first

The single largest improvement in the entire project came from switching the waypoints shaping function from `WaypointSumDistanceShaping` to `WaypointDistanceShaping` (NWS): a one-line change that moved `ep_rew_mean` from -260 to -100 with no other modification. This was only discovered late in the experiment sequence after significant compute had been spent on hyperparameter tuning. With more compute, we would have run a dedicated shaping ablation at the very start. A well-designed shaping function is more impactful than any hyperparameter sweep and it is cheaper to test changes early than late.

5.4.2 Replace SAC with TQC or TD3 on sparse tasks

SAC's Q-overestimation collapse was reproducible across all seeds and configurations on the waypoints task. Rather than continuing to tune SAC's hyperparameters, an alternative off-policy algorithm should have been tried earlier. TQC (Truncated Quantile Critics) is a direct extension of SAC that uses distributional critics and explicitly drops the top quantiles to counteract overestimation. TD3 avoids overestimation via delayed policy updates and target policy smoothing and its deterministic policy eliminates the entropy instability seen with SAC's auto-tuning on mode 0. Both would be worth comparing against SAC on the waypoints task before committing to a curriculum strategy.

5.4.3 More seeds and proper statistical evaluation

Most curriculum intermediate stages were run with a single seed due to compute constraints. This makes it impossible to be sure that an initialisation is not just due to luck. Following Agarwal et al. (2021)[1], we would run a minimum of 5 seeds per configuration and report interquartile mean (IQM) with stratified bootstrap confidence intervals rather than raw per-seed curves. This is especially important for the waypoints task, where high variance between seeds (as seen in the PPO stage 4 results: 800-900 range across 3 seeds) makes point estimates misleading.

5.4.4 Stricter WandB run organisation

Run naming and grouping conventions were established mid-project, resulting in early runs with inconsistent group tags, missing seeds in some groups and re-runs that are difficult to distinguish from intentional ablations. This made cross-run comparisons on WandB manually intensive: identifying which runs belonged to the same sweep required reading config files rather than filtering by group. With more compute and time, we would enforce a fixed naming schema (`algo_task_hyperparam_mode_seed`) and group structure from the first run and use WandB sweeps for any grid search rather than launching individual runs. This would allow automated aggregation of seed results and direct IQM computation within the WandB interface.

5.4.5 Extend the mode 6 curriculum to the full task

The mode 6 curriculum stalled at 2 targets in a 30 m dome (+190-200 reward). The same staged transfer that worked for mode 0 (small dome → full dome, more targets) was never fully applied to mode 6 due to time constraints. With more compute, we would have continued the chain: 2 targets at 30 m → 3 targets at 80 m → 4 targets at 150 m, matching the mode 0 curriculum design. This would produce a directly comparable mode 6 final model and a cleaner answer to the flight-mode comparison question.

6 Dogfight Tournament Strategy

6.1 Overview

For the dogfight environment, we used a self-play reinforcement learning framework combined with a curriculum warmup. We chose this approach to solve the issues of sparse rewards and early training instability. At the very beginning of training, our agents were quite unskilled and tended to crash into the ground immediately, long before they could learn any meaningful combat strategies. By using a curriculum, we allowed the agents to learn basic flight and stability first. This approach ensured that the agent could actually stay in the air and participate in a fight before it had to worry about winning a dogfight.

6.2 Curriculum Learning Strategy

We trained a PPO agent over a total of 1 million environment steps, utilizing 8 parallel dogfight environments to accelerate the training process. The training regime is divided into two distinct phases to facilitate a curriculum learning approach:

- **Phase 1: Heuristic Warm-up (100,000 steps):** The agent initially trains against a stable heuristic opponent that follows a predictable flight path. This provides a consistent target for learning pursuit and aiming mechanics, generating a reliable reward signal that would be difficult to obtain against a highly dynamic or random opponent.

- **Phase 2: Self-Play (900,000 steps):** After the warm-up, the system transitions to self-play. The opponent policy is initially randomised, but every 50,000 steps, the current policy is frozen and added to the opponent pool. This creates an automated curriculum where the agent faces increasingly more competent versions of itself, preventing overfitting to a fixed strategy.

6.3 Submitted Agent

The agent submitted to the tournament is the PPO policy obtained at the end of the self-play phase (1M total steps: 100k heuristic warmup + 900k self-play with a 50k-step opponent pool refresh). We do not provide a quantitative analysis of dogfight performance in this report. The agent's competitive standing is determined directly by the tournament Elo leaderboard.

7 Beyond the Required Experiments

Several components of this work go beyond the minimum requirement of training and comparing two algorithms. We list them here for clarity.

7.1 From-scratch PPO implementation

In addition to the SB3 PPO used for the main experiments, we implemented PPO from scratch in PyTorch (clipped surrogate objective, GAE, separate actor/critic optimisers, full-batch updates).

7.2 Curriculum learning

The mode 0 waypoints result (800-900 reward across 3 seeds on the full evaluation environment) was only obtained through a staged curriculum: 1 target in a 30 m dome $\rightarrow$ 3 targets in a 30 m dome $\rightarrow$ 3 targets in a 150 m dome $\rightarrow$ 4 targets in a 150 m dome. Each stage initialises from the previous stage's policy. Direct training on the full task fails for both PPO and SAC.

7.3 Reward shaping ablation

We compared two potential-based shaping functions (`WaypointSumDistanceShaping` vs `WaypointDistanceShaping`) and found that next waypoint shaping produced the largest improvement, lifting `ep_rew_mean` from -260 to -100 on mode 6 with no other modification.

7.4 Hyperparameter sweeps

On hover we swept $\gamma \in \{0.98, 0.99\}$, PPO $n_s \in \{2048, 4096\}$, and SAC UTD $\in \{3, 4\}$, across both flight modes, with 2 seeds per configuration. The sweep revealed two non-trivial failure modes (γ=0.99 entropy instability on SAC mode 0; PPO collapse at γ=0.98, n_s=2048 on mode 6) that informed the configurations used downstream.

7.5 Self-play with policy pool for dogfight

The dogfight agent uses heuristic warmup followed by self-play against a frozen opponent pool refreshed every 50k steps, rather than training against a single fixed opponent (see Section 6).

8 Conclusion

This project compared PPO and SAC across three drone control tasks of increasing difficulty: stabilisation (hover), sparse-reward navigation (waypoints) and adversarial combat (dogfight). The central finding is that the on-policy / off-policy trade-off does not have a uniform answer across tasks.

On hover, both algorithms converge to the same final reward ($\approx$1100), but SAC is 5-10$\times$ more sample-efficient than PPO, a clear win for off-policy learning when the reward signal is dense. On waypoints, SAC fails on every configuration we tried due to Q-value overestimation collapse on the sparse reward, while PPO with a staged curriculum and next waypoint reward shaping reaches 800-900 reward across 3 seeds on the full evaluation environment. The same replay buffer that makes SAC efficient on hover becomes the mechanism of its failure on waypoints, because the buffer's transition distribution diverges from the policy's actual visitation distribution when high-reward states are rare.

Beyond the algorithm comparison, two methodological choices proved decisive on the hard task: curriculum learning (none of our direct training runs on the full waypoints task produced any navigation behaviour) and reward shaping selection (next waypoint shaping moved `ep_rew_mean` from -260 to -100 in a one-line change). Both took meaningful compute to discover. We noted in the limitations that, with hindsight, a shaping ablation should have come before any hyperparameter tuning.

For the dogfight tournament we submit a PPO agent trained with heuristic warmup and self-play against a refreshed opponent pool.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, 2021.